`N` from its parent `P`, and adding the two new spheres to `P`. Obviously, P could also now have M+1 children! Long story short, we need to backtrack to the root, and we can only stop if we get to a node that had less than `M` children, or if we do get to the root: if we have to split the root, then we will create a new root with just 2 children, and the height of the tree will grow by 1 (and that's the only case this can happen.

Listing 10.6 shows an implementation of the `insert` method:

- The tree traversal, equivalent to the `searchParentLeaf` method, appears at lines #6 and #7;
- Case (1) above is handled at line #3, where we return `null` to let the caller know there is no further action required;
- Case (2) corresponds to lines #5 and #12 in the pseudocode, also resulting in the method returning `null`;
- Case (3), which clearly is the most complicated option, is coded in lines #13 and #14;
- Backtracking is handled at lines #8 to #12.

Figures 10.14 and 10.15 illustrates the third case above, where we insert a point in a leaf that already contains `M` points. At a high level, insertion in Ss-trees follows B-tree's algorithm for `insert`: the only difference is in the way we split nodes (in B-trees the list of elements is just split in two halves); of course, in B-trees also links to children and ordering are handled differently, as we have seen in section 10.2.